

1. Visão geral da aplicação

Tasker é uma aplicação web de gerenciamento de tarefas (to-do list) com autenticação de usuários via JWT. Permite cadastro, login, criação, listagem, atualização e exclusão de tarefas associadas ao usuário autenticado.

A aplicação foi empacotada em **três contêineres Docker** orquestrados por **Docker Compose**, seguindo o princípio de isolamento de rede: somente o frontend é exposto ao host. Backend e banco de dados ficam acessíveis apenas dentro da rede interna do Compose, comunicando-se pelo DNS do Docker (nome do serviço).

Funcionalidades

- Registro e autenticação de usuário (JWT).
- CRUD de tarefas vinculadas ao usuário autenticado.
- Migrações Prisma aplicadas automaticamente no start do backend.
- Frontend com SSR via Next.js, proxy interno para o backend usando rewrites.

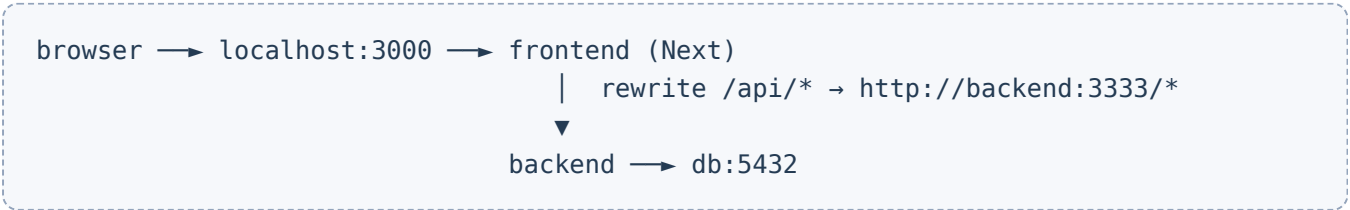
2. Arquitetura e contêineres

Serviço	Imagem / Build	Porta host	Acesso externo
db	postgres:16-alpine	—	não
backend	build ./tasker-backend	—	não
frontend	build ./tasker-frontend	3000	sim

Comunicação interna sempre pelo nome do serviço (DNS interno do Docker):

- `backend` → `db` via `postgresql://...@db:5432/tasker`
- `frontend` (Node server) → `backend` via `http://backend:3333`, usado no rewrite do Next (`next.config.mjs`)

O browser fala apenas com `localhost:3000`. Requisições a `/api/*` batem no servidor Next, que proxia internamente para `backend:3333`. Nenhuma porta do backend é publicada no host.



3. Tecnologias utilizadas

Camada	Tecnologia	Função
Frontend	Next.js 14 (React)	SSR, roteamento, proxy via rewrites
Backend	Fastify (Node.js 20)	API REST, autenticação JWT
ORM	Prisma	Migrações e acesso ao banco
Banco	PostgreSQL 16 (Alpine)	Persistência de usuários e tarefas
Linguagem	TypeScript	Backend e frontend
Build	Dockerfile multi-stage	deps → builder → runner
Orquestração	Docker Compose v2	Subir os três serviços

4. Pré-requisitos

- Docker Engine 24+
- Docker Compose v2 (`docker compose`)
- Git

5. Manual de instalação

5.1 Clonar repositório

```
git clone <url-do-repo> tasker
cd tasker
```

5.2 Configurar variáveis de ambiente

```
cp .env.example .env
```

Ajuste `SECRET_JWT` e credenciais do banco se desejar. Valores padrão funcionam para desenvolvimento.

```
POSTGRES_USER=docker
POSTGRES_PASSWORD=docker
POSTGRES_DB=tasker

DATABASE_URL=postgresql://docker:docker@db:5432/tasker?schema=public
SECRET_JWT=supersecretjwt
BACKEND_PORT=3333

FRONTEND_PORT=3000
```

5.3 Subir a stack

```
docker compose up --build
```

Em background:

```
docker compose up -d --build
```

5.4 Acessar

- Frontend: <http://localhost:3000>
- Backend: não acessível do host — apenas via proxy do frontend em [/api/*](#) (ex.: <http://localhost:3000/api/user>).

As migrações do Prisma rodam automaticamente no start do backend (`prisma migrate deploy`).

6. Estrutura do projeto

```
.
├── compose.yml
├── .env
├── .env.example
├── tasker-backend/
│   ├── Dockerfile
│   ├── .dockerignore
│   └── src/, prisma/, ...
└── tasker-frontend/
    ├── Dockerfile
    ├── .dockerignore
    └── app/, components/, ...
```

7. Detalhes dos Dockerfiles

7.1 Backend (multi-stage)

```
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npx prisma generate
RUN npm run build
```

```

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
RUN apk add --no-cache openssl
COPY --from=deps /app/node_modules ./node_modules
COPY --from=builder /app/build ./build
COPY --from=builder /app/prisma ./prisma
COPY --from=builder /app/node_modules/.prisma ./node_modules/.prisma
COPY package.json ./
EXPOSE 3333
CMD ["sh", "-c", "npx prisma migrate deploy && node build/server.js"]

```

7.2 Frontend (multi-stage)

```

FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/.next ./next
COPY --from=builder /app/public ./public
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json ./package.json
COPY --from=builder /app/next.config.mjs ./next.config.mjs
EXPOSE 3000
CMD ["npm", "run", "start"]

```

8. compose.yml

```

services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]

```

```
interval: 5s
timeout: 5s
retries: 10
```

backend:

```
build: ./tasker-backend
depends_on:
  db:
    condition: service_healthy
environment:
  DATABASE_URL: ${DATABASE_URL}
  SECRET_JWT: ${SECRET_JWT}
  PORT: ${BACKEND_PORT}
expose:
  - "${BACKEND_PORT}"
```

frontend:

```
build:
  context: ./tasker-frontend
depends_on:
  - backend
ports:
  - "${FRONTEND_PORT}:3000"
```